

WEBSCRAPING

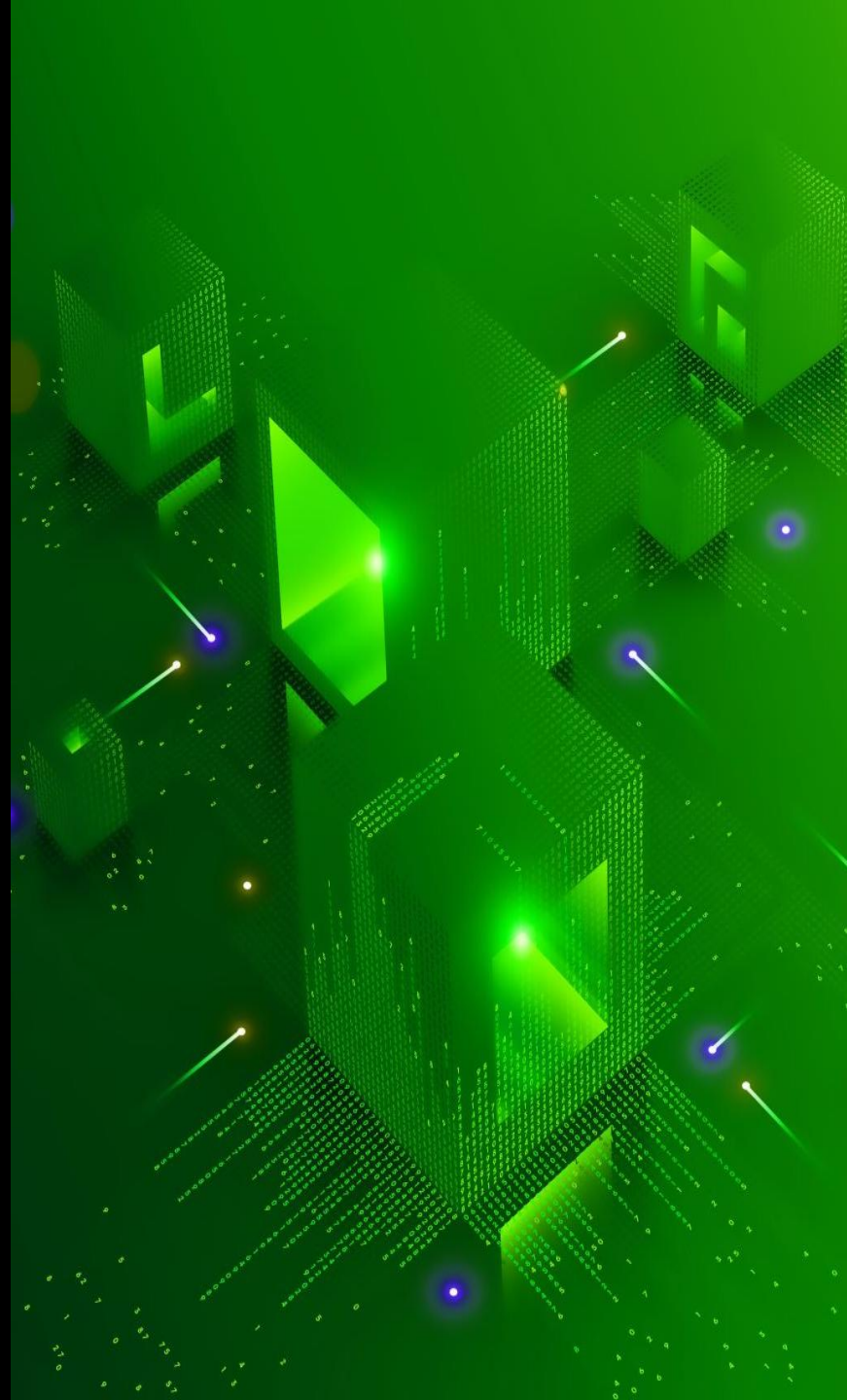
USING BEAUTIFUL SOUP

Introduction

- Beautiful Soup is a powerful Python library used for web scraping purposes to extract data from HTML and XML files.
- Beautiful Soup creates a parse tree from webpage source code, making it easy to navigate, search, and modify the parse tree.
- It provides Pythonic idioms for searching within the document. For example, you can search for a tag by its name, class, id, and other tags.
- Install Beautiful soup* - From command line type:

```
python –m pip install bs4
```

* This assumes python is in the environment path



Why Scrape the Web?



Get data that has no API — prices, listings, filings and public records only exist as web pages.



Scale beyond copy-paste — one script collects thousands of rows in minutes, on a schedule.



Build datasets — feed dashboards, research and machine-learning models with fresh, real-world data.



Monitor change over time — track competitor pricing, job postings or policy updates automatically.



Example Tag Scraping

Find all the Table tags

```
html_doc = '''<html><body>
<table>
<tr><td>Row 1, Cell 1</td><td>Row 1, Cell 2</td></tr>
<tr><td>Row 2, Cell 1</td><td>Row 2, Cell 2</td></tr>
</table>
</body></html>
'''
soup = BeautifulSoup(html_doc, 'html.parser')
rows = soup.find_all('tr')
for row in rows:
    cells = row.find_all('td')
    for cell in cells:
        print(cell.text)
```

Features



Web Scraping: Extract data from web pages for data analysis, machine learning, and other applications.



Data Cleaning: Clean and parse the data from web pages to store in structured formats like databases or CSV files.



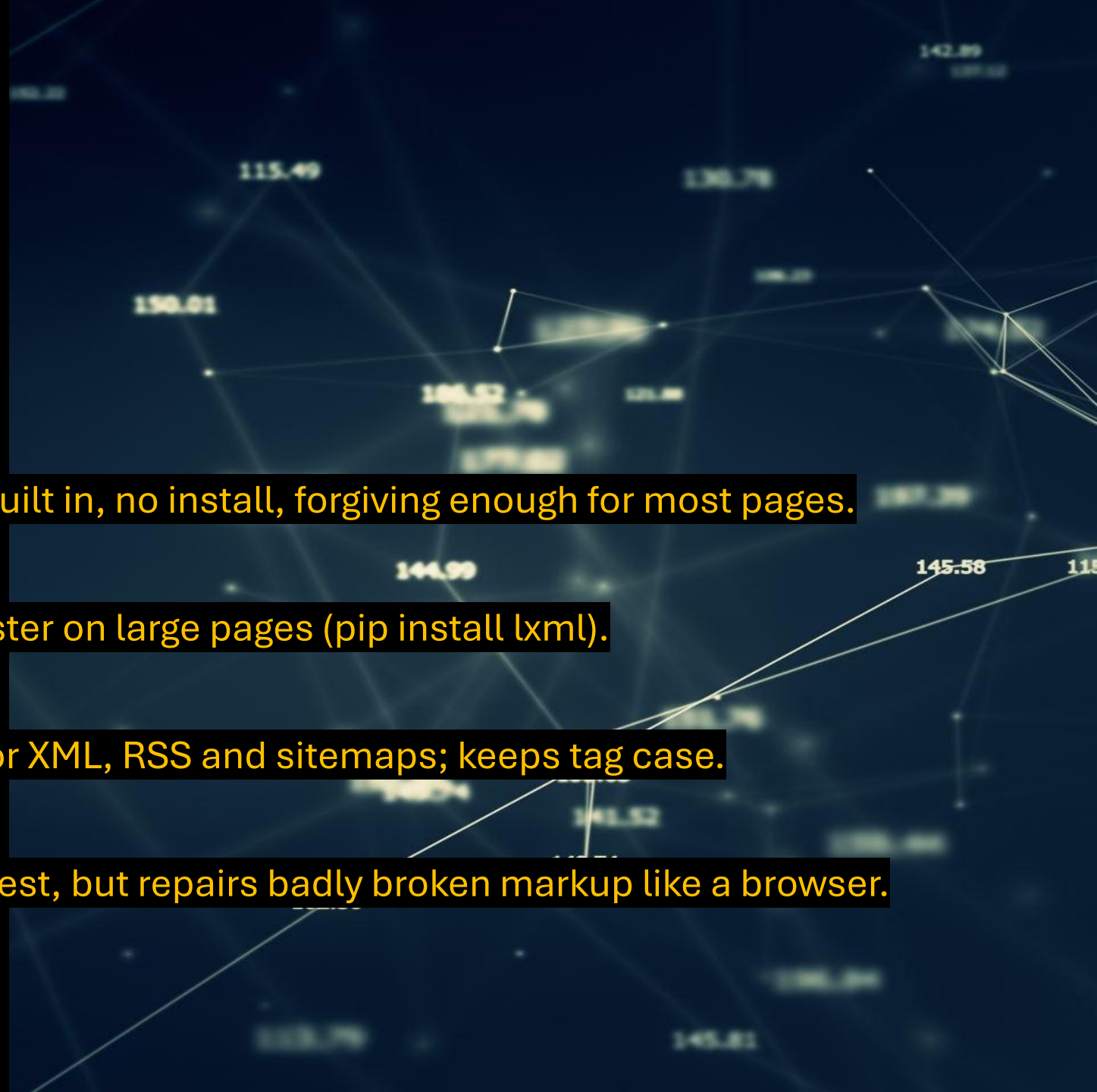
Automation: collect data where no API exists. Check robots.txt and the site's terms first, identify your bot in the User-Agent, and rate-limit.



Limits: BS4 only parses HTML you already have — it does not run JavaScript. For JS-rendered pages use Playwright or Selenium, then pass the HTML to BS4.



Parsing HTML/XML



- BeautifulSoup(html, 'html.parser') — built in, no install, forgiving enough for most pages.
- BeautifulSoup(html, 'lxml') — much faster on large pages (pip install lxml).
- BeautifulSoup(xml, 'xml') — required for XML, RSS and sitemaps; keeps tag case.
- BeautifulSoup(html, 'html5lib') — slowest, but repairs badly broken markup like a browser.

Fetching Pages

- r = requests.get(url, headers={'User-Agent':'my-scraper/1.0'}, timeout=10)
- r.raise_for_status()
- Check robots.txt and the site's terms first; prefer an official API if one exists.
- Sleep ~1s between requests and cache pages so re-runs don't re-hit the server.
- Retry on 429 / 503 with a longer wait.



Navigating the Parse Tree

soup.tag — first match only: soup.title, soup.a. Chains, e.g. soup.body.h1. Returns None (not an error) if absent.

soup.find_all('a', href=True, limit=10) — returns a list; add class_='btn' or string='Next'. Slow on huge pages, so use limit.

soup.find('div', {'id':'main'}) — first match or None. Always guard: if tag is not None before reading .text.



Searching the Parse Tree



find_all(attrs={'data-id':'42'}) — needed for hyphenated or reserved attributes. Use class_ for class, since class is a Python keyword.



find('img', attrs={'alt':re.compile('logo', re.I)}) — values accept a string, regex, list, or True (attribute exists).



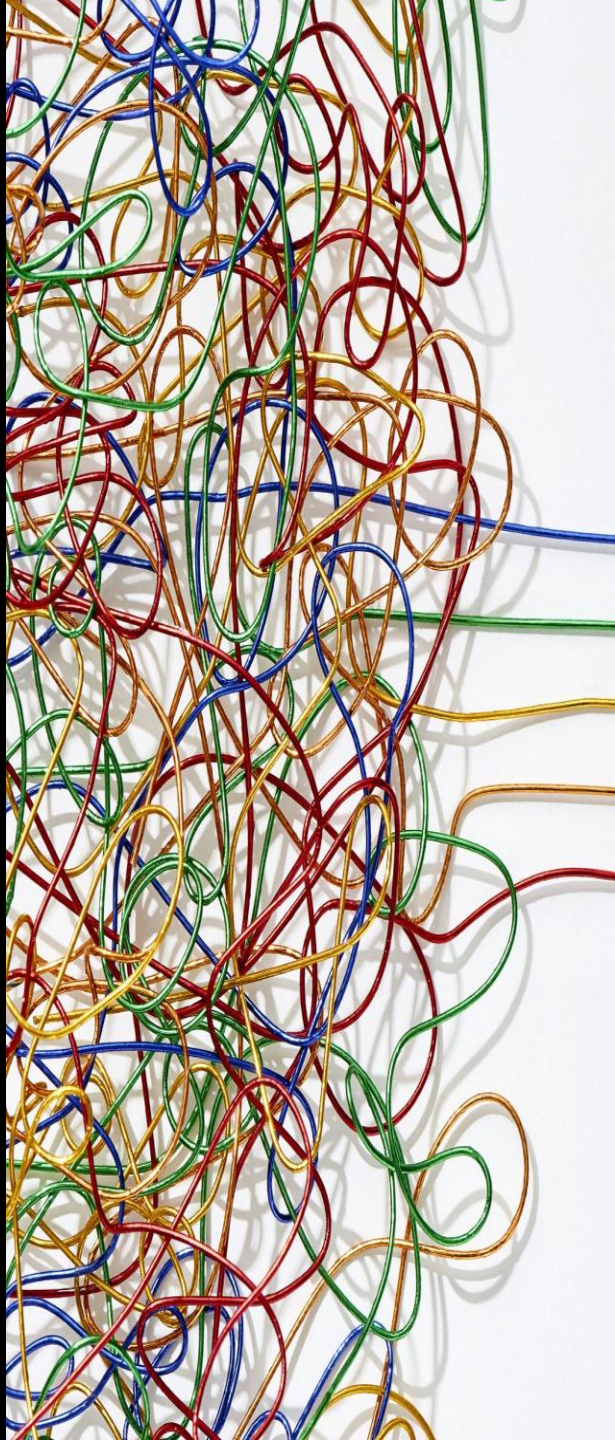
soup.select('table.data tr td:nth-of-type(2)') — CSS selectors handle descendants and nesting; select_one() returns a single element.

Modifying the Parse Tree

tag.string is None when a tag has mixed children — prefer tag.get_text(strip=True, separator=' ') for reliable text.

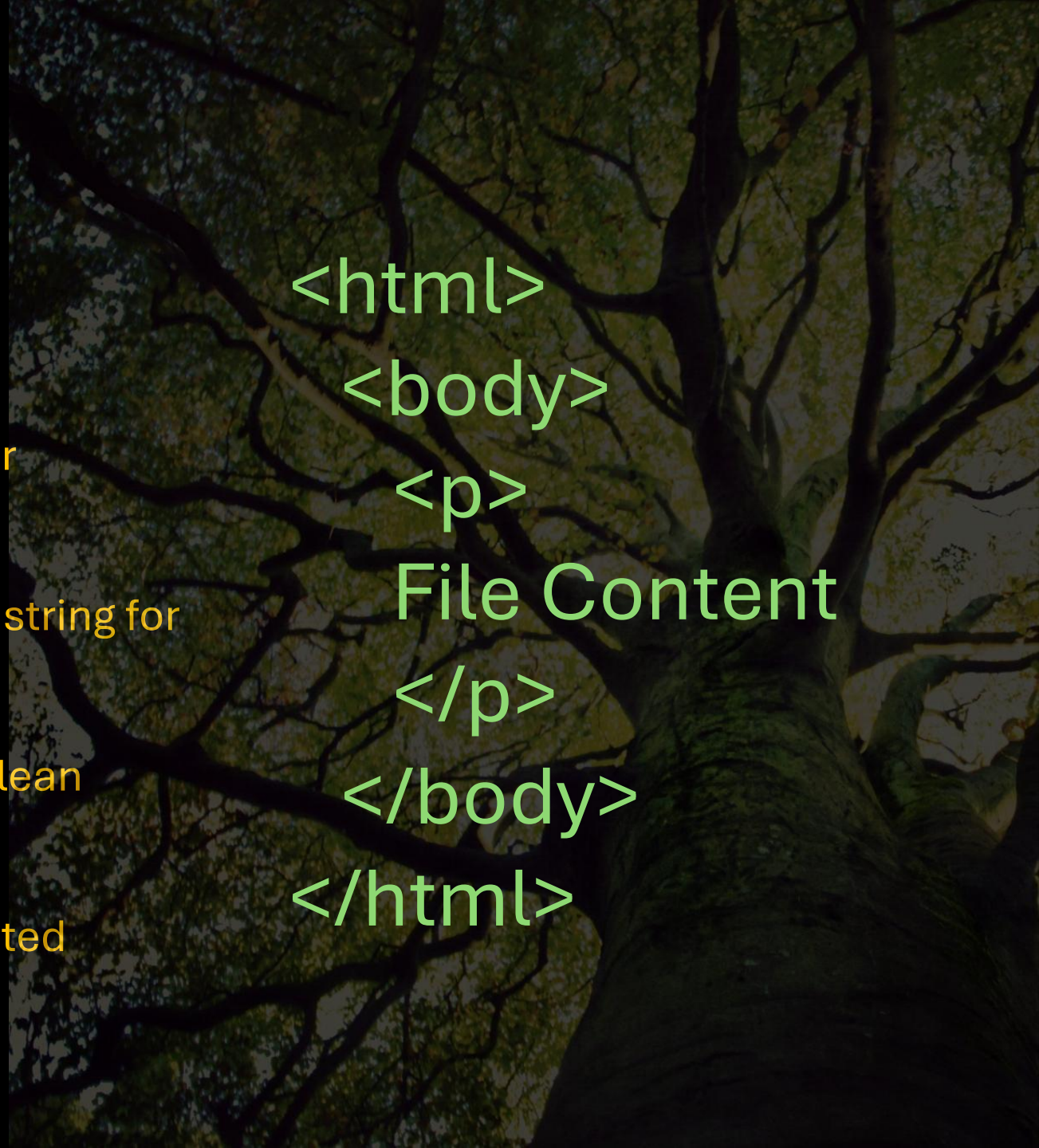
tag['href'] = '/new' sets a value; tag.get('href', '') avoids KeyError; del tag['style'] removes an attribute.

tag.decompose() destroys a tag; tag.extract() returns it for reuse. Strip <script> and <style> before extracting text.



Outputting HTML/XML

- soup.prettify() — indented HTML, ideal for eyeballing structure while you develop.
- str(soup) / str(tag) — back to a raw HTML string for saving or diffing.
- tag.get_text(separator=' ', strip=True) — clean visible text with no tags.
- Open files with encoding='utf-8' or accented characters will break on Windows.



Navigating the Tree

Tag	tag.contents / tag.children include whitespace strings — filter with tag.find_all(recursive=False).
Tag	tag.parent walks up one level; tag.find_parent('table') jumps straight to an ancestor by name.
Tag	next_sibling often returns '\n' — use find_next_sibling('td') to skip whitespace nodes.

Why Beautiful Soup Beats Regex

Structure vs. characters: BS4 parses HTML into a tree and understands tags, attributes and nesting; regex only sees a flat string.

Broken markup: browsers and BS4 repair unclosed or misnested tags — a regex silently matches the wrong thing.

Readable intent: soup.select('table.data tr td:nth-of-type(2)') vs. a 200-character unreadable pattern.

Robust to change: attribute order, whitespace and extra classes break regex; BS4 keeps working.

Use both: BS4 finds the tags, regex filters the values — soup.find_all('a', href=re.compile(r'\.pdf$')).

Example using RE and BS4

Find all links with URLs matching a certain regex pattern

```
html_doc = """
<html><body>
<a href="http://example.com/page1">Link 1</a>
<a href="https://example.com/page2">Link 2</a>
</body></html>
"""

soup = BeautifulSoup(html_doc, 'html.parser')
links = soup.find_all('a', href=re.compile(r'^http://'))
for link in links:
    print(link.get('href'))
```

Data Extraction

Fetch & parse

`r = requests.get(url, timeout=10)`

`soup = BeautifulSoup(r.text, 'html.parser')`

Extract rows

`cells = row.find_all('td')`

`cells[0].get_text().strip()`

Paginate politely

`urljoin(base, a['href'])`

`visited set + sleep ~1s`

Save results

`pd.DataFrame(rows).to_csv(...)`

or

`pd.read_html(str(table))[0]`

Data Cleaning

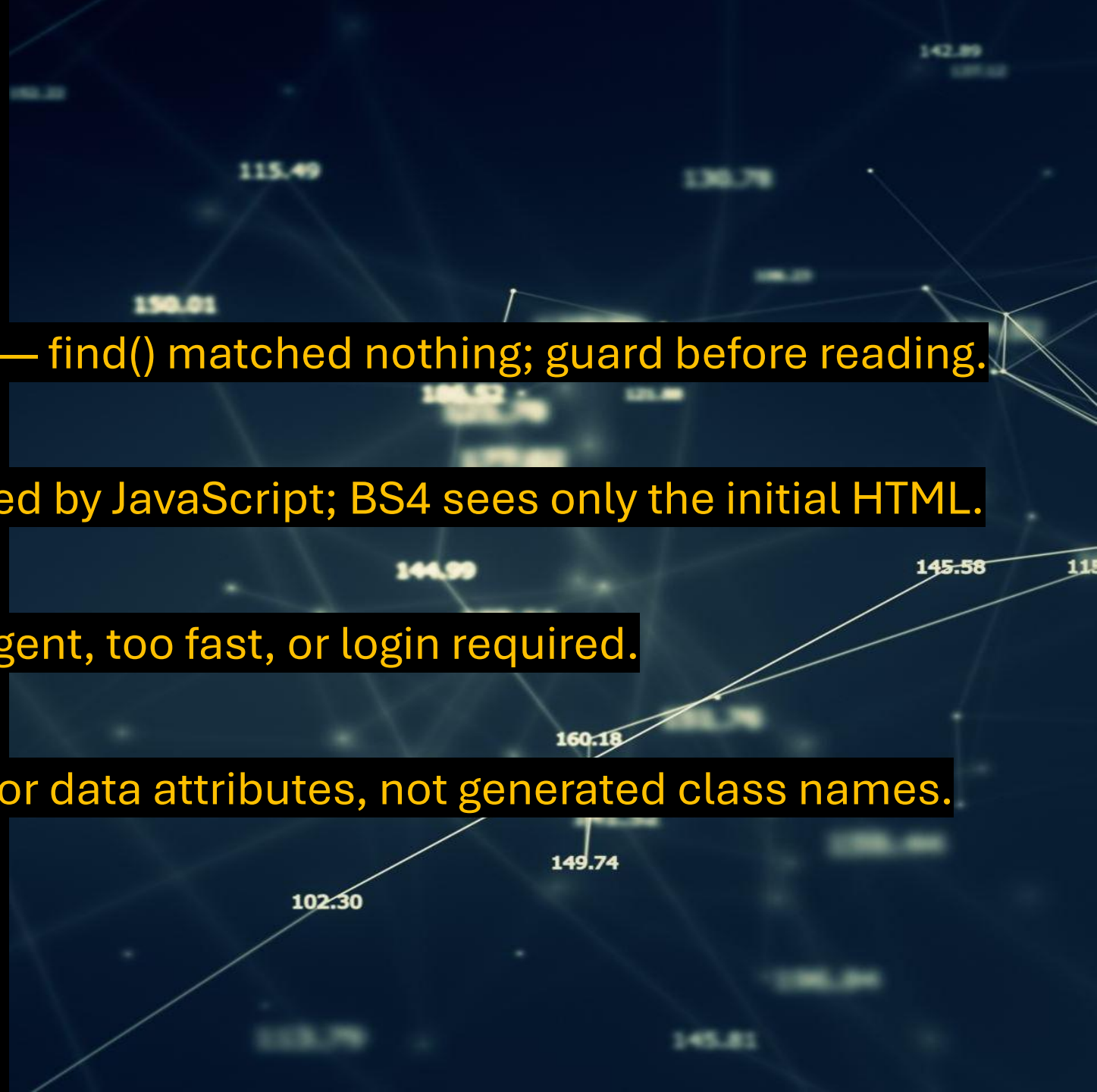
Normalize text: .get_text(strip=True), then re.sub(r'\s+',' ',s) and s.replace('\xa0',' ') to kill non-breaking spaces.

Cast types: float(price.strip('$').replace(',','')) and pd.to_datetime(date_str, errors='coerce') so bad rows become NaT, not crashes.

Deduplicate and validate: df.drop_duplicates(), df.dropna(subset=['price']), and log rows you dropped so silent data loss is visible.

Common Pitfalls

- NoneType has no attribute 'text' — find() matched nothing; guard before reading.
- Empty results — content rendered by JavaScript; BS4 sees only the initial HTML.
- 403 / blocked — missing User-Agent, too fast, or login required.
- Brittle selectors — anchor on id or data attributes, not generated class names.



Webscraping Tutorials



- HTML & CSS selectors — w3schools.com/html/
- Beautiful Soup walkthrough — realpython.com/beautiful-soup-web-scraper-python/
- Official BS4 docs — crummy.com/software/BeautifulSoup/bs4/doc/
- Safe practice sites — books.toscrape.com, quotes.toscrape.com